Relatório Técnico — Matrix Calculator

1. Introdução

O projeto Matrix Calculator consiste em uma aplicação de linha de comando, desenvolvida em Python, cujo objetivo é permitir a criação, visualização e manipulação de matrizes numéricas. O programa busca solucionar a necessidade de executar operações básicas de álgebra linear — soma, subtração, multiplicação matricial, multiplicação por escalar e transposição — de forma simples e interativa, sem depender de bibliotecas matemáticas externas ou interfaces gráficas.

2. Decisões de implementação

A arquitetura do projeto foi organizada em três módulos com responsabilidades bem definidas, seguindo o princípio de separação de responsabilidades:

- **main.py:** concentra a interação com o usuário, incluindo o menu principal implementado com a estrutura match/case (recurso disponível a partir do Python 3.10) e as funções de coleta de dados de entrada.
- **matrix.py:** define a classe MatrixOperations, que encapsula toda a lógica matemática das operações sobre matrizes.
- **display_matrix.py:** define a classe DisplayMatrix, dedicada exclusivamente à formatação e exibição das matrizes no terminal.

A classe MatrixOperations foi projetada para aceitar uma segunda matriz (matrixB) como parâmetro opcional no construtor, permitindo que a mesma classe fosse reutilizada tanto para operações binárias (soma, subtração, multiplicação) quanto para operações unárias sobre uma única matriz (multiplicação por escalar e transposição), evitando a criação de uma classe separada apenas para esses dois casos. As operações matemáticas foram implementadas manualmente, por meio de laços aninhados, sem o uso de bibliotecas como NumPy — decisão tomada para manter o projeto restrito à biblioteca padrão do Python e reforçar, do ponto de vista didático, o entendimento do funcionamento interno de cada operação matricial. Optou-se também por usar match/case em vez de uma cadeia de if/elif no menu principal, por tornar a leitura das opções mais direta.
Outra decisão relevante foi a validação explícita de dimensões antes da execução das operações de soma, subtração e multiplicação, levantando exceções ValueError com mensagens específicas quando as matrizes são incompatíveis, em vez de deixar o programa falhar com um erro genérico de índice.

3. Dificuldades encontradas

Durante o desenvolvimento, um dos principais desafios foi lidar com entradas inválidas do usuário no terminal: como todos os valores (dimensões, elementos das matrizes e opções do menu) são lidos com input() e convertidos com int(), qualquer caractere não numérico digitado por engano interrompia a execução com um ValueError não tratado nas primeiras versões do programa. Outra dificuldade foi garantir que operações entre matrizes de dimensões incompatíveis não gerassem erros pouco informativos, como IndexError, o que exigiu adicionar verificações de dimensão antes de cada operação binária. Também houve dificuldade inicial em decidir como reaproveitar a classe MatrixOperations tanto para operações com uma matriz quanto com duas, sem duplicar código entre elas.

4. Soluções adotadas

Para o problema de entradas inválidas, todo o laço principal do menu em main.py foi envolvido em um bloco try/except ValueError, de forma que qualquer erro de conversão numérica — ou erro de dimensão levantado pelas operações — é capturado e exibido ao usuário como uma mensagem amigável, sem encerrar o programa. Para o problema das dimensões incompatíveis, cada método de MatrixOperations que exige compatibilidade (add, subtract e multiply) passou a validar as dimensões

das matrizes logo no início da execução, levantando um ValueError com mensagem descritiva antes de qualquer tentativa de acesso aos elementos. Já o problema de reaproveitamento de código foi resolvido tornando matrixB um parâmetro opcional no construtor de MatrixOperations, permitindo instanciar a classe apenas com matrixA quando a operação (scalar_multiply ou transpose) não depende de uma segunda matriz.

5. Considerações finais

O projeto atinge seu objetivo de oferecer uma calculadora de matrizes funcional via terminal, com separação clara entre lógica de negócio (MatrixOperations), apresentação (DisplayMatrix) e interação com o usuário (main.py), além de um tratamento de erros consistente para as principais falhas de uso. Como limitações observadas, destacam-se a ausência de testes automatizados, a ausência de validação de que a matriz informada seja retangular (todas as linhas com o mesmo número de colunas) no momento da criação, e o fato de as opções de multiplicação por escalar e transposição operarem apenas sobre a matriz A. Como possíveis melhorias futuras, sugerem-se a criação de testes unitários para os métodos de MatrixOperations, a validação da regularidade das matrizes já na entrada de dados, e a extensão do menu para permitir as mesmas operações unárias também sobre a matriz B.